

Code:

```
import math

from collections import Counter

DATASET = [{"Outlook": "Sunny", "Temperature": "Hot", "Humidity": "High",
"Wind": "Weak", "PlayTennis": "No"}, {"Outlook": "Sunny", "Temperature": "Hot",
"Humidity": "High", "Wind": "Strong", "PlayTennis": "No"}, {"Outlook": "Overcast",
"Temperature": "Hot", "Humidity": "High", "Wind": "Weak", "PlayTennis":
"Yes"}, {"Outlook": "Rain", "Temperature": "Mild", "Humidity": "High", "Wind":
"Weak", "PlayTennis": "Yes"}, {"Outlook": "Rain", "Temperature": "Cool",
"Humidity": "Normal", "Wind": "Weak", "PlayTennis": "Yes"}, {"Outlook": "Rain",
"Temperature": "Cool", "Humidity": "Normal", "Wind": "Strong", "PlayTennis":
"No"}, {"Outlook": "Overcast", "Temperature": "Cool", "Humidity": "Normal", "Wind":
"Strong", "PlayTennis": "Yes"}, {"Outlook": "Sunny", "Temperature": "Mild",
"Humidity": "High", "Wind": "Weak", "PlayTennis": "No"}, {"Outlook": "Sunny",
"Temperature": "Cool", "Humidity": "Normal", "Wind": "Weak", "PlayTennis":
"Yes"}, {"Outlook": "Rain", "Temperature": "Mild", "Humidity": "Normal", "Wind":
"Weak", "PlayTennis": "Yes"}, {"Outlook": "Sunny", "Temperature": "Mild",
"Humidity": "Normal", "Wind": "Strong", "PlayTennis": "Yes"}, {"Outlook": "Overcast",
"Temperature": "Mild", "Humidity": "High", "Wind": "Strong", "PlayTennis":
"Yes"}, {"Outlook": "Overcast", "Temperature": "Hot", "Humidity": "Normal", "Wind":
"Weak", "PlayTennis": "Yes"}, {"Outlook": "Rain", "Temperature": "Mild",
"Humidity": "High", "Wind": "Strong", "PlayTennis": "No"},
]

FEATURES = ["Outlook", "Temperature", "Humidity", "Wind"]

TARGET = "PlayTennis"

def entropy(data: list[dict]) -> float:

    if not data:

        return 0.0

    counts = Counter(row[TARGET] for row in data)
```

```

total = len(data)

result = 0.0

for count in counts.values():

    p = count / total

    result -= p * math.log2(p)

return result

def information_gain(data: list[dict], feature: str) -> float:

    total = len(data)

    base_h = entropy(data)

    weighted = 0.0

    subsets: dict[str, list] = {}

    for row in data:

        val = row[feature]

        subsets.setdefault(val, []).append(row)

    for subset in subsets.values():

        weighted += (len(subset) / total) * entropy(subset)

    return base_h - weighted

class Node:

    def __init__(self, feature: str | None = None, label: str | None = None):

        self.feature = feature

        self.label = label

        self.children: dict[str, "Node"] = {}

    def is_leaf(self) -> bool:

        return self.label is not None

```

```

def __repr__(self) -> str:

    if self.is_leaf():

        return f"Leaf({self.label!r})"

    return f"Node(feature={self.feature!r}, branches={list(self.children)})"

def id3(data: list[dict], features: list[str], depth: int = 0) -> Node:

    labels = [row[TARGET] for row in data]

    if len(set(labels)) == 1:

        print(f'{' ' * depth}[Leaf] All examples → {labels[0]!r}')

        return Node(label=labels[0])

    if not features:

        majority = Counter(labels).most_common(1)[0][0]

        print(f'{' ' * depth}[Leaf] No features left → majority = {majority!r}')

        return Node(label=majority)

    gains = {f: information_gain(data, f) for f in features}

    best = max(gains, key=gains.get)

    print(f'{' ' * depth}[Node] Splitting on {best!r} (IG = {gains[best]:.4f})')

    print(f'{' ' * depth}    Gains: "

        + ", ".join(f'{f}={g:.4f}' for f, g in sorted(gains.items()))

    node = Node(feature=best)

    remaining_feats = [f for f in features if f != best]

    feature_values = {row[best] for row in data}

    for value in sorted(feature_values):

        subset = [row for row in data if row[best] == value]

        print(f'{' ' * depth} Branch {best}={value!r} ({len(subset)} examples)")

```

```

    if not subset:

        majority = Counter(labels).most_common(1)[0][0]

        node.children[value] = Node(label=majority)

    else:

        node.children[value] = id3(subset, remaining_feats, depth + 1)

    return node

def predict(node: Node, sample: dict) -> str:

    if node.is_leaf():

        return node.label

    value = sample.get(node.feature)

    child = node.children.get(value)

    if child is None:

        child = next(iter(node.children.values()))

    return predict(child, sample)

def print_tree(node: Node, indent: int = 0, branch_label: str = "") -> None:

    prefix = " " * indent

    connector = f"[{branch_label}] " if branch_label else ""

    if node.is_leaf():

        print(f"{prefix}{connector}→ {node.label}")

    else:

        print(f"{prefix}{connector}{node.feature}?")

        for value, child in sorted(node.children.items()):

            print_tree(child, indent + 1, branch_label=value)

def main():

```

```

sep = "=" * 60

print(sep)

print("STEP 1 — Entropy of the full dataset")

print(sep)

h = entropy(DATASET)

label_counts = Counter(row[TARGET] for row in DATASET)

print(f" Labels : {dict(label_counts)}")

print(f" Total : {len(DATASET)} examples")

print(f" H(S) = {h:.4f} bits\n")

print(sep)

print("STEP 2 — Information Gain for each feature")

print(sep)

for feat in FEATURES:

    ig = information_gain(DATASET, feat)

    print(f" IG({feat:<12}) = {ig:.4f}")

print()

print(sep)

print("STEP 3 — Building the ID3 Tree (verbose trace)")

print(sep)

tree = id3(DATASET, FEATURES)

print()

print(sep)

print("STEP 4 — Decision Tree Structure")

print(sep)

```

```

print_tree(tree)

print()

print(sep)

print("STEP 5 — Predictions on new samples")

print(sep)

test_samples = [{"Outlook": "Sunny", "Temperature": "Cool", "Humidity": "Normal",
"Wind": "Strong"}, {"Outlook": "Rain", "Temperature": "Mild", "Humidity": "High",
"Wind": "Strong"}, {"Outlook": "Overcast", "Temperature": "Hot", "Humidity":
"Normal", "Wind": "Weak"},]

for sample in test_samples:

    result = predict(tree, sample)

    attrs = ", ".join(f'{k}={v}' for k, v in sample.items())

    print(f' [{attrs}]')

    print(f' → PlayTennis = {result}\n')

print(sep)

print("STEP 6 — Training Accuracy")

print(sep)

correct = sum(1 for row in DATASET if predict(tree, row) == row[TARGET])

print(f' Correct : {correct}/{len(DATASET)}')

print(f' Accuracy: {correct/len(DATASET)*100:.1f}%\n')

if __name__ == "__main__":

    main()

```

Output:

STEP 1 – Entropy of the full dataset

```
Labels : {'No': 5, 'Yes': 9}
Total  : 14 examples
H(S)   = 0.9403 bits
```

STEP 2 – Information Gain for each feature

```
IG(Outlook    ) = 0.2467
IG(Temperature) = 0.0292
IG(Humidity   ) = 0.1518
IG(Wind       ) = 0.0481
```

STEP 3 – Building the ID3 Tree (verbose trace)

```
[Node] Splitting on 'Outlook' (IG = 0.2467)
      Gains: Humidity=0.1518, Outlook=0.2467, Temperature=0.0292, Wind=0.0481
      Branch Outlook='Overcast' (4 examples)
      [Leaf] All examples → 'Yes'
      Branch Outlook='Rain' (5 examples)
      [Node] Splitting on 'Wind' (IG = 0.9710)
            Gains: Humidity=0.0200, Temperature=0.0200, Wind=0.9710
            Branch Wind='Strong' (2 examples)
            [Leaf] All examples → 'No'
            Branch Wind='Weak' (3 examples)
            [Leaf] All examples → 'Yes'
      Branch Outlook='Sunny' (5 examples)
      [Node] Splitting on 'Humidity' (IG = 0.9710)
            Gains: Humidity=0.9710, Temperature=0.5710, Wind=0.0200
            Branch Humidity='High' (3 examples)
            [Leaf] All examples → 'No'
            Branch Humidity='Normal' (2 examples)
            [Leaf] All examples → 'Yes'
```

STEP 4 – Decision Tree Structure

```
Outlook?
[Overcast] → Yes
[Rain] Wind?
  [Strong] → No
  [Weak] → Yes
[Sunny] Humidity?
  [High] → No
  [Normal] → Yes
```

STEP 5 – Predictions on new samples

```
[Outlook=Sunny, Temperature=Cool, Humidity=Normal, Wind=Strong]
  → PlayTennis = Yes

[Outlook=Rain, Temperature=Mild, Humidity=High, Wind=Strong]
  → PlayTennis = No

[Outlook=Overcast, Temperature=Hot, Humidity=Normal, Wind=Weak]
  → PlayTennis = Yes
```

STEP 6 – Training Accuracy

```
Correct : 14/14
Accuracy: 100.0%
```

Conclusion:

The ID3 algorithm was implemented successfully to make decision tree.

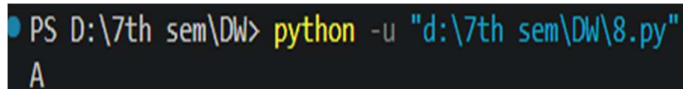
Code:

```
import numpy as np
from collections import Counter

def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((np.array(point1) - np.array(point2))**2))

def knn_predict(training_data, training_labels, test_point, k):
    distances = []
    for i in range(len(training_data)):
        dist = euclidean_distance(test_point, training_data[i])
        distances.append((dist, training_labels[i]))
    distances.sort(key=lambda x: x[0])
    k_nearest_labels = [label for _, label in distances[:k]]
    return Counter(k_nearest_labels).most_common(1)[0][0]

training_data = [[1, 2], [2, 3], [3, 4], [6, 7], [7, 8]]
training_labels = ['A', 'A', 'A', 'B', 'B']
test_point = [4, 5]
k = 3
prediction = knn_predict(training_data, training_labels, test_point, k)
print(prediction)
```

Output:

```
PS D:\7th sem\Dw> python -u "d:\7th sem\Dw\8.py"
A
```

Conclusion:

The KNN algorithm was implemented successfully to find nearest neighbor.

Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

data = load_iris()

X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = GaussianNB()

model.fit(X_train, y_train)

probabilities = model.predict_proba(X_test)

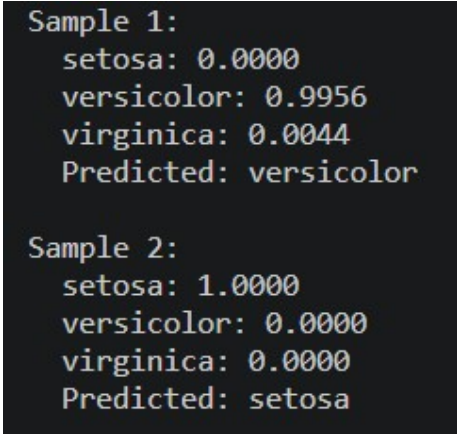
for i, probs in enumerate(probabilities[:2]):

    print(f"Sample {i+1}:")

    for cls, prob in zip(data.target_names, probs):

        print(f" {cls}: {prob:.4f}")

    print(f" Predicted: {data.target_names[probs.argmax()]}\\n")
```

Output:

```
Sample 1:
setosa: 0.0000
versicolor: 0.9956
virginica: 0.0044
Predicted: versicolor

Sample 2:
setosa: 1.0000
versicolor: 0.0000
virginica: 0.0000
Predicted: setosa
```

Conclusion:

The Naïve Bayes algorithm was implemented successfully to classify sample datasets.

Code:

```
from sklearn.svm import SVC

from sklearn.preprocessing import StandardScaler

import numpy as np

X = np.array([[5000, 25, 10],[15000, 45, 50],[8000, 30, 20],[20000, 50, 60],[3000, 22,
5],[18000, 48, 55],[7000, 28, 15],[25000, 55, 70],])

y = np.array([0, 1, 0, 1, 0, 1, 0, 1])

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

model = SVC(kernel='rbf', probability=True)

model.fit(X_scaled, y)

new_customer = np.array([[12000, 40, 35]])

new_scaled = scaler.transform(new_customer)

prediction = model.predict(new_scaled)

probability = model.predict_proba(new_scaled)

labels = {0: "Low Value Customer", 1: "High Value Customer"}

print("=== SVM for Data Warehouse Customer Segmentation ===\n")

print(f"New Customer Data:")

print(f" Sales Amount    : $12,000")

print(f" Customer Age      : 40")

print(f" Purchase Frequency: 35 times\n")

print(f"Prediction   : {labels[prediction[0]]}")

print(f"Probability    : Low Value = {probability[0][0]:.4f} | High Value =
{probability[0][1]:.4f}")

print(f"\nSupport Vectors Count: {model.n_support_}")
```

Output:

```
=== SVM for Data Warehouse Customer Segmentation ===  
  
New Customer Data:  
Sales Amount      : $12,000  
Customer Age      : 40  
Purchase Frequency: 35 times  
  
Prediction       : High Value Customer  
Probability      : Low Value = 0.4022 | High Value = 0.5978  
  
Support Vectors Count: [2 2]
```

Conclusion:

The SVM algorithm was implemented successfully to classify sample datasets.

Code:

```
import numpy as np

def sigmoid(x):

    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):

    return x * (1 - x)

X = np.array([[2, 60], [4, 75], [6, 80], [8, 90], [1, 50], [3, 65], [7, 85], [5, 70]])

y = np.array([[0], [0], [1], [1], [0], [0], [1], [1]])

X = X / np.array([10, 100])

np.random.seed(42)

w1 = np.random.randn(2, 4)

w2 = np.random.randn(4, 1)

b1 = np.zeros((1, 4))

b2 = np.zeros((1, 1))

lr = 0.5

epochs = 5000

for epoch in range(epochs):

    # Forward Pass

    z1 = np.dot(X, w1) + b1

    a1 = sigmoid(z1)

    z2 = np.dot(a1, w2) + b2

    a2 = sigmoid(z2)

    error = y - a2

    d2 = error * sigmoid_derivative(a2)
```

```

d1 = np.dot(d2, w2.T) * sigmoid_derivative(a1)

w2 += np.dot(a1.T, d2) * lr

b2 += np.sum(d2, axis=0) * lr

w1 += np.dot(X.T, d1) * lr

b1 += np.sum(d1, axis=0) * lr

if epoch % 1000 == 0:

    loss = np.mean(np.square(error))

    print(f'Epoch {epoch:>5} | Loss: {loss:.4f}')

print("\n=== Predictions ===")

print(f'{'Hours':>6}      {'Attendance':>10}      {'Actual':>8}      {'Predicted':>10}
{'Probability':>12}')

print("-" * 50)

for i in range(len(X)):

    z1 = np.dot(X[i], w1) + b1

    a1 = sigmoid(z1)

    z2 = np.dot(a1, w2) + b2

    prob = sigmoid(z2)[0][0]

    pred = 1 if prob >= 0.5 else 0

```

Output:

```

Epoch    0 | Loss: 0.2953
Epoch 1000 | Loss: 0.0070
Epoch 2000 | Loss: 0.0021
Epoch 3000 | Loss: 0.0011
Epoch 4000 | Loss: 0.0007

=== Predictions ===
Hours Attendance Actual Predicted Probability

```

Conclusion:

The backpropagation was implemented successfully to classify datasets.

Code:

```
import numpy as np

from statsmodels.stats.contingency_tables import mcnemar

y_true = np.array([1,0,1,1,0,1,0,0,1,1,0,1,0,1,1,0,1])
y_clf1 = np.array([1,0,1,0,0,1,0,0,1,1,0,1,0,1,1,1,0,1])
y_clf2 = np.array([1,0,0,1,0,1,0,1,1,0,0,1,0,1,0,0,1,0,0])

a = np.sum((y_clf1 == y_true) & (y_clf2 == y_true))
b = np.sum((y_clf1 == y_true) & (y_clf2 != y_true))
c = np.sum((y_clf1 != y_true) & (y_clf2 == y_true))
d = np.sum((y_clf1 != y_true) & (y_clf2 != y_true))

table = np.array([[a, b], [c, d]])

result = mcnemar(table, exact=True)

print("==== McNemar's Test ====\n")

print(f"Contingency Table:")

print(f"          CLF2 Correct  CLF2 Wrong")

print(f" CLF1 Correct |   {a}   |   {b}   |")

print(f" CLF1 Wrong  |   {c}   |   {d}   |")

print(f"\nCLF1 Accuracy : {np.mean(y_clf1 == y_true):.2f}")

print(f"CLF2 Accuracy : {np.mean(y_clf2 == y_true):.2f}")

print(f"\nMcNemar Statistic : {result.statistic:.4f}")

print(f"P-Value       : {result.pvalue:.4f}")

print(f"\nConclusion: {'Significant difference between classifiers.' if result.pvalue < 0.05
else 'No significant difference between classifiers.'}")
```

Output:

```
PS D:\7th sem\DW> python -u "d:\7th sem\DW\12.py"
=== McNemar's Test ===

Contingency Table:
          CLF2 Correct  CLF2 Wrong
CLF1 Correct |      14      |      4      |
CLF1 Wrong  |       2      |      0      |

CLF1 Accuracy : 0.90
CLF2 Accuracy : 0.80

McNemar Statistic : 2.0000
P-Value           : 0.6875

Conclusion: No significant difference between classifiers.
```

Conclusion:

The McNemar's test was implemented successfully to compare two classifiers.

Code:

```
import numpy as np

X = np.array([
    [20, 30], [22, 35], [25, 28],
    [80, 90], [85, 88], [78, 92],
    [50, 55], [52, 58], [48, 53]
])

K = 3

np.random.seed(42)

centroids = X[np.random.choice(len(X), K, replace=False)]

def assign_clusters(X, centroids):
    distances = np.array([[np.linalg.norm(x - c) for c in centroids] for x in X])
    return np.argmin(distances, axis=1)

def update_centroids(X, labels):
    return np.array([X[labels == k].mean(axis=0) for k in range(K)])

for iteration in range(10):
    labels = assign_clusters(X, centroids)
    new_centroids = update_centroids(X, labels)
    if np.allclose(centroids, new_centroids):
        print(f"Converged at iteration {iteration}")
        break
    centroids = new_centroids

print("\n=== K-Means Clustering Results ===\n")

print(f'{"Point":<8} {"Spending":>10} {"Income":>10} {"Cluster":>10}')
```

```

print("-" * 42)

for i, (point, label) in enumerate(zip(X, labels)):

    print(f'{i+1:<8} {point[0]:>10} {point[1]:>10} {label+1:>10}')

print(f'\nFinal Centroids:')

for i, c in enumerate(centroids):

    print(f' Cluster {i+1}: Spending={c[0]:.1f}, Income={c[1]:.1f}')

```

Output:

```

Converged at iteration 1

=== K-Means Clustering Results ===

Point      Spending      Income      Cluster
-----
1           20           30           2
2           22           35           2
3           25           28           2
4           80           90           3
5           85           88           3
6           78           92           3
7           50           55           1
8           52           58           1
9           48           53           1

Final Centroids:
Cluster 1: Spending=50.0, Income=55.3
Cluster 2: Spending=22.3, Income=31.0
Cluster 3: Spending=81.0, Income=90.0

```

Conclusion:

The K means was used successfully to implement clustering of the sample datasets.

Code:

```
import numpy as np

X = np.array([ [20, 30], [22, 35], [25, 28], [80, 90], [85, 88], [78, 92], [50, 55], [52, 58], [48, 53]])

K = 3

np.random.seed(42)

def kmeans_plus_plus(X, K):

    centroids = [X[np.random.randint(len(X))]]

    for _ in range(K - 1):

        distances = np.array([min(np.linalg.norm(x - c)**2 for c in centroids) for x in X])

        probs = distances / distances.sum()

        centroids.append(X[np.random.choice(len(X), p=probs)])

    return np.array(centroids)

def assign_clusters(X, centroids):

    distances = np.array([[np.linalg.norm(x - c) for c in centroids] for x in X])

    return np.argmin(distances, axis=1)

def update_centroids(X, labels):

    return np.array([X[labels == k].mean(axis=0) for k in range(K)])

centroids = kmeans_plus_plus(X, K)

print("==== Initial Centroids (K-Means++) ====")

for i, c in enumerate(centroids):

    print(f" Centroid {i+1}: {c}")

for iteration in range(10):

    labels = assign_clusters(X, centroids)
```

```

new_centroids = update_centroids(X, labels)

if np.allclose(centroids, new_centroids):

    print(f"\nConverged at iteration {iteration}")

    break

centroids = new_centroids

print("\n=== K-Means++ Clustering Results ===\n")

print(f'{"Point":<8} {"Spending":>10} {"Income":>10} {"Cluster":>10}')

print("-" * 42)

for i, (point, label) in enumerate(zip(X, labels)):

    print(f' {i+1:<8} {point[0]:>10} {point[1]:>10} {label+1:>10}')

print(f"\nFinal Centroids:")

for i, c in enumerate(centroids):

    print(f' Cluster {i+1}: Spending={c[0]:.1f}, Income={c[1]:.1f}')

```

Output:

```

PS D:\7th sem\DW> python -u "d:\7th sem\DW\14.py"
=== Initial Centroids (K-Means++) ===
Centroid 1: [50 55]
Centroid 2: [78 92]
Centroid 3: [20 30]

Converged at iteration 1

=== K-Means++ Clustering Results ===

Point      Spending    Income    Cluster
-----
1           20         30         3
2           22         35         3
3           25         28         3
4           80         90         2
5           85         88         2
6           78         92         2
7           50         55         1
8           52         58         1
9           48         53         1

Final Centroids:
Cluster 1: Spending=50.0, Income=55.3
Cluster 2: Spending=81.0, Income=90.0
Cluster 3: Spending=22.3, Income=31.0

```

Conclusion:

The K means++ was used successfully to implement clustering of the sample datasets.

Code:

```
import numpy as np

X = np.array([ [20, 30], [22, 35], [25, 28],[80, 90], [85, 88], [78, 92],[50, 55], [52, 58], [48, 53]])

K = 3

np.random.seed(42)

def compute_cost(X, labels, medoids):

    return sum(np.linalg.norm(X[i] - medoids[labels[i]]) for i in range(len(X)))

def assign_clusters(X, medoids):

    distances = np.array([[np.linalg.norm(x - m) for m in medoids] for x in X])

    return np.argmin(distances, axis=1)

medoid_indices = np.random.choice(len(X), K, replace=False)

medoids = X[medoid_indices]

print("=== Initial Medoids ===")

for i, m in enumerate(medoids):

    print(f" Medoid {i+1}: {m}")

for iteration in range(10):

    labels = assign_clusters(X, medoids)

    current_cost = compute_cost(X, labels, medoids)

    improved = False

    for k in range(K):

        for i in range(len(X)):

            if i in medoid_indices:

                continue

            new_medoid_indices = medoid_indices.copy()
```

```

new_medoid_indices[k] = i

new_medoids = X[new_medoid_indices]

new_labels = assign_clusters(X, new_medoids)

new_cost = compute_cost(X, new_labels, new_medoids)

if new_cost < current_cost:

    medoid_indices = new_medoid_indices

    medoids = new_medoids

    current_cost = new_cost

    improved = True

if not improved:

    print(f'Converged at iteration {iteration + 1}')

    break

labels = assign_clusters(X, medoids)

print("\n==== K-Medoids Clustering Results ==== \n")

print(f'{"Point":<8} {"Spending":>10} {"Income":>10} {"Cluster":>10}')

print("-" * 42)

for i, (point, label) in enumerate(zip(X, labels)):

    print(f'{i+1:<8} {point[0]:>10} {point[1]:>10} {label+1:>10}')

print(f'\nFinal Medoids (Actual Data Points):')

for i, m in enumerate(medoids):

    print(f' Medoid {i+1}: Spending={m[0]}, Income={m[1]}')

print(f'\nTotal Cost: {current_cost:.4f}')

```

Output:

```
PS D:\7th sem\DW> python -u "d:\7th sem\DW\15.py"
=== Initial Medoids ===
Medoid 1: [52 58]
Medoid 2: [22 35]
Medoid 3: [78 92]
Converged at iteration 2

=== K-Medoids Clustering Results ===

Point    Spending    Income    Cluster
-----
1         20         30        2
2         22         35        2
3         25         28        2
4         80         90        3
5         85         88        3
6         78         92        3
7         50         55        1
8         52         58        1
9         48         53        1

Final Medoids (Actual Data Points):
Medoid 1: Spending=50, Income=55
Medoid 2: Spending=20, Income=30
Medoid 3: Spending=80, Income=90

Total Cost: 25.4179
```

Conclusion:

The K-medoids was implemented successfully and the datasets were clustered successfully.

Code:

```
import numpy as np

X = np.array([ [20, 30], [22, 35], [25, 28], [80, 90], [85, 88], [78, 92], [50, 55], [52, 58],
               [48, 53]])

def euclidean(a, b):

    return np.linalg.norm(a - b)

def cluster_distance(c1, c2, X):

    return max(euclidean(X[i], X[j]) for i in c1 for j in c2)

clusters = [ {i} for i in range(len(X))]

history = []

print("=== Agglomerative Clustering (Complete Linkage) ===\n")

while len(clusters) > K:= 3:

    min_dist = float('inf')

    merge_i, merge_j = 0, 1

    for i in range(len(clusters)):

        for j in range(i + 1, len(clusters)):

            dist = cluster_distance(clusters[i], clusters[j], X)

            if dist < min_dist:

                min_dist = dist

                merge_i, merge_j = i, j

    merged = clusters[merge_i] | clusters[merge_j]

    history.append((clusters[merge_i], clusters[merge_j], round(min_dist, 2)))

    clusters = [c for k, c in enumerate(clusters) if k != merge_i and k != merge_j]

    clusters.append(merged)

print("Merge History:")
```

```

for step, (c1, c2, dist) in enumerate(history):

    print(f" Step {step+1}: Cluster {set(c1)} + {set(c2)} -> Distance={dist}")

labels = np.zeros(len(X), dtype=int)

for cluster_id, cluster in enumerate(clusters):

    for idx in cluster:

        labels[idx] = cluster_id + 1

print(f"\n=== Final Clusters (K={K}) ===\n")

print(f"{'Point':<8} {'Spending':>10} {'Income':>10} {'Cluster':>10}")

print("-" * 42)

for i, (point, label) in enumerate(zip(X, labels)):

    print(f"{'i+1':<8} {'point[0]:>10} {'point[1]:>10} {'label:>10}")

```

Output:

```

PS D:\7th sem\DW> python -u "d:\7th sem\DW\16.py"
=== Agglomerative Clustering (Complete Linkage) ===

Merge History:
Step 1: Cluster {3} + {5} -> Distance=2.83
Step 2: Cluster {6} + {8} -> Distance=2.83
Step 3: Cluster {0} + {1} -> Distance=5.39
Step 4: Cluster {7} + {8, 6} -> Distance=6.4
Step 5: Cluster {2} + {0, 1} -> Distance=7.62
Step 6: Cluster {4} + {3, 5} -> Distance=8.06

=== Final Clusters (K=3) ===

Point      Spending      Income      Cluster
-----
1           20           30           2
2           22           35           2
3           25           28           2
4           80           90           3
5           85           88           3
6           78           92           3
7           50           55           1
8           52           58           1
9           48           53           1

```

Conclusion:

The agglomerative algorithm was implemented successfully and sample datasets were clustered successfully.

Code:

```
import numpy as np

X = np.array([[20, 30], [22, 35], [25, 28],[80, 90], [85, 88], [78, 92],[50, 55], [52, 58], [48,
53]])

K = 3

def euclidean(a, b):

    return np.linalg.norm(a - b)

def assign_kmeans(X, indices, k=2):

    pts = X[indices]

    centroids = pts[np.random.choice(len(pts), k, replace=False)]

    for _ in range(100):

        labels = np.argmin([[euclidean(p, c) for c in centroids] for p in pts], axis=1)

        new_centroids = np.array([pts[labels == i].mean(axis=0) if np.any(labels == i) else
centroids[i] for i in range(k)])

        if np.allclose(centroids, new_centroids):

            break

        centroids = new_centroids

    return labels

def split_cluster(X, indices):

    labels = assign_kmeans(X, indices, k=2)

    return [list(np.array(indices)[labels == i]) for i in range(2)]

def cluster_diameter(X, indices):

    pts = X[indices]

    return max(euclidean(pts[i], pts[j]) for i in range(len(pts)) for j in range(i+1, len(pts))) if
len(pts) > 1 else 0
```

```

np.random.seed(42)

clusters = [list(range(len(X)))]

history = []

print("=== Divisive Clustering ===\n")

while len(clusters) < K:

    diameters = [cluster_diameter(X, c) for c in clusters]

    split_idx = np.argmax(diameters)

    to_split = clusters.pop(split_idx)

    split1, split2 = split_cluster(X, to_split)

    clusters.extend([split1, split2])

    history.append((to_split, split1, split2, round(diameters[split_idx], 2)))

print("Split History:")

for step, (original, s1, s2, diam) in enumerate(history):

    print(f" Step {step+1}: Cluster {original} split into {s1} and {s2} (diameter={diam})")

labels = np.zeros(len(X), dtype=int)

for cluster_id, cluster in enumerate(clusters):

    for idx in cluster:

        labels[idx] = cluster_id + 1

print(f"\n=== Final Clusters (K={K}) ===\n")

print(f"{'Point':<8} {'Spending':>10} {'Income':>10} {'Cluster':>10}")

print("-" * 42)

for i, (point, label) in enumerate(zip(X, labels)):

    print(f"{i+1:<8} {point[0]:>10} {point[1]:>10} {label:>10}")

print(f"\nFinal Cluster Members:")

```

for i, cluster in enumerate(clusters):

print(f" Cluster {i+1}: Points {cluster}")

Output:

```
PS D:\7th sem\DW> python -u "d:\7th sem\DW\tempCodeRunnerFile.py"
=== Divisive Clustering ===

Split History:
  Step 1: Cluster [0, 1, 2, 3, 4, 5, 6, 7, 8] split into [np.int64(3),
64(2)] (diameter=87.11)
  Step 2: Cluster [np.int64(3), np.int64(4), np.int64(5), np.int64(6),
4(4), np.int64(5)] (diameter=50.93)

=== Final Clusters (K=3) ===

Point      Spending      Income      Cluster
-----
1           20          30          1
2           22          35          1
3           25          28          1
4           80          90          3
5           85          88          3
6           78          92          3
7           50          55          2
8           52          58          2
9           48          53          2

Final Cluster Members:
  Cluster 1: Points [np.int64(0), np.int64(1), np.int64(2)]
  Cluster 2: Points [np.int64(6), np.int64(7), np.int64(8)]
  Cluster 3: Points [np.int64(3), np.int64(4), np.int64(5)]
PS D:\7th sem\DW>
```

Conclusion:

The divisive algorithm was implemented successfully to cluster the sample datasets.